🌀 Procedural Maze Generator – Documentation

📦 Overview

The **Procedural Maze Generator** is a lightweight Unity tool that allows you to generate random mazes directly in the editor or at runtime.
It's perfect for prototyping, puzzle games, dungeon crawlers, and AI pathfinding tests.

⚙️ Installation

1. Import the package into your Unity project.
2. Drag the `MazeRenderer` prefab (or script) into your scene.
3. Assign wall and floor prefabs in the inspector.

🛠️ How to Use

1. Basic Setup

- Add the **MazeRenderer** component to an empty GameObject.
- Assign prefabs:
    - **Wall Prefab** – used for maze walls.
    - **Floor Prefab** – used for ground tiles.

2. Settings in Inspector

- **Rows (Y):** Number of rows in the maze.
- **Columns (X):** Number of columns in the maze.

- **Cell Size:** Spacing between each cell.
- **Seed:** Enter a seed for reproducible mazes. Leave at 0 for randomness.
- **Generate On Start:** Automatically generate the maze at runtime.
- **Exit Placement:** Choose where the entrance/exit should be placed. Options:
    - `OppositeCorners` → Entrance at bottom-left, exit at top-right
    - `RandomEdges` → Random positions on maze edges
    - `Custom` → Manually assign entrance & exit

3. Generating a Maze

- Press the **Generate Maze** button in the inspector (Editor mode).
- Or call `mazeRenderer.GenerateMaze();` in code.

🎨 Customization

- **Materials & Prefabs:** Replace wall/floor prefabs with your own models.
- **Scale & Size:** Adjust `Cell Size` to change spacing.
- **Themes:** You can make stone, sci-fi, or cartoon mazes by swapping prefabs.

📜 Example Code Usage

```
using UnityEngine;

public class MazeExample : MonoBehaviour
{
    public MazeRenderer maze;

    void Start()
    {
```

```
        // Generate a new 15x15 maze at runtime
        maze.rows = 15;
        maze.cols = 15;
        maze.cellSize = 2f;
        maze.GenerateMaze();
    }
}
```

🚪 Entrance & Exit

- The system automatically creates an **entrance and exit** based on the chosen placement option.
- If using **Custom**, you can set entrance/exit positions manually by calling:

```
maze.SetCustomEntrance(new Vector2Int(0, 5));
maze.SetCustomExit(new Vector2Int(14, 5));
```

⚡ Performance Notes

- Designed for lightweight performance.
- Works with any prefab size or shape.
- For large mazes (>100x100), consider pooling or runtime generation.

🧩 Use Cases

- Puzzle & Escape games
- Procedural dungeon generation
- Roguelike level design
- AI navigation/pathfinding tests

📑 Support

For bug reports or feature requests, please contact lazifdev@gmail.com.